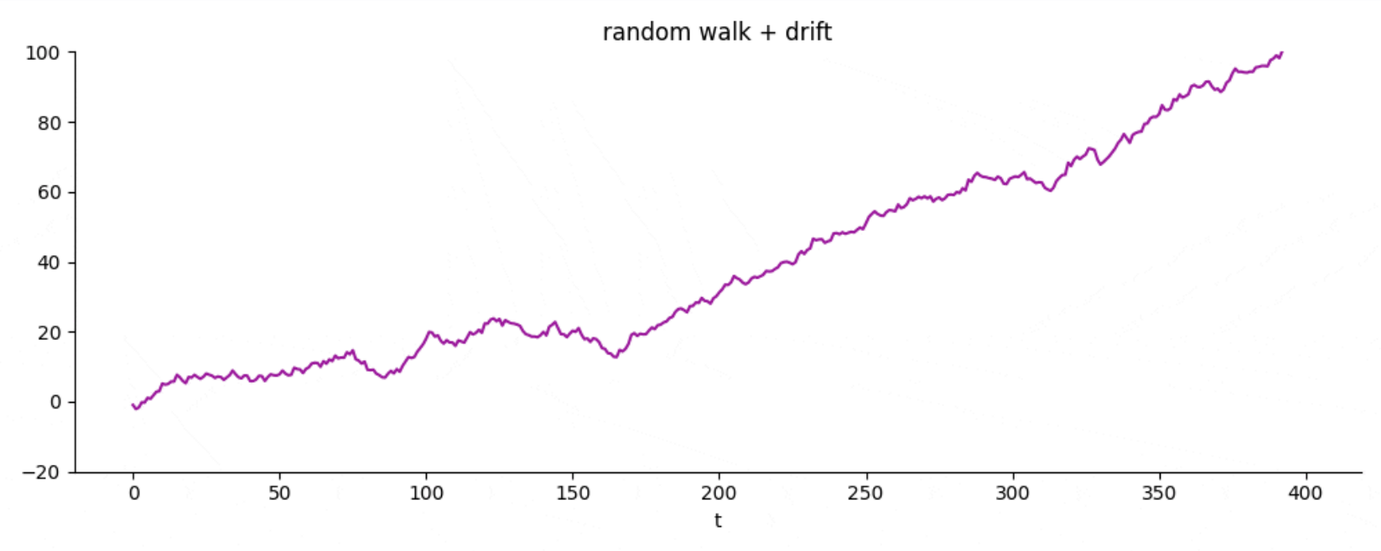
Forecastability & how to increase it

WARM UP QUESTION 1/1

Consider the random process $x_t = c + x_{t-1} + \epsilon$, also known as random walk with drift.

1. What's the mean of the process at time t?
2. Is the process ergodic?
3. Can you make it stationary and ergodic by taking first differences?



100 realizations of a random walk with drift and c=0.2 over 400 steps

Increasing forecastability by enforcing stationarity

Manipulating the time series before forecasting can help make it stationary.

In forecasting, only **causal**, **invertible** transformations are admissible.

Causal: we are not using information from step t to transform previous timesteps

Invertible: we must be able to nullify the applied transformations, so that we can apply a pipeline transform → forecast → anti-transform

1. Distributional transformations

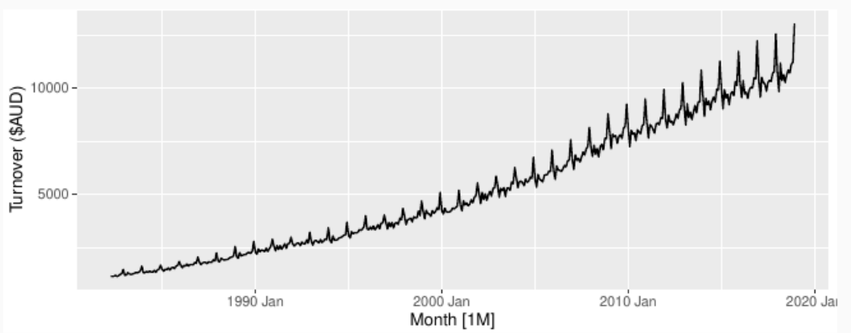
Why?

- making data distribution more Gaussian generally increases performance when fitting with a sum of squares loss

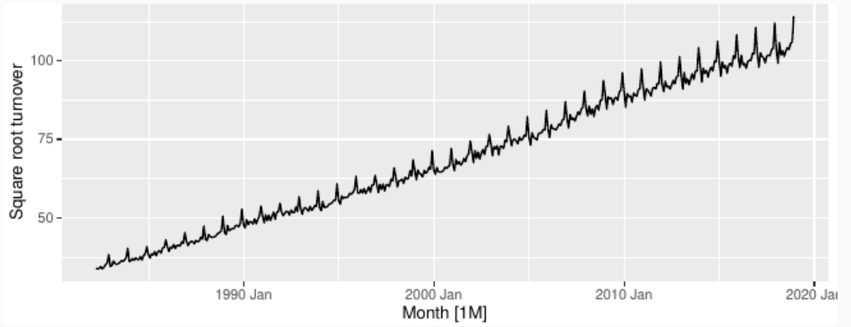
How?

- by simple functions
- by parametric transformations

Simple transformations are easier to explain and work well enough. log1p() can be useful for data with zeros. Choosing logs is a simple way to force forecasts to be positive



Original

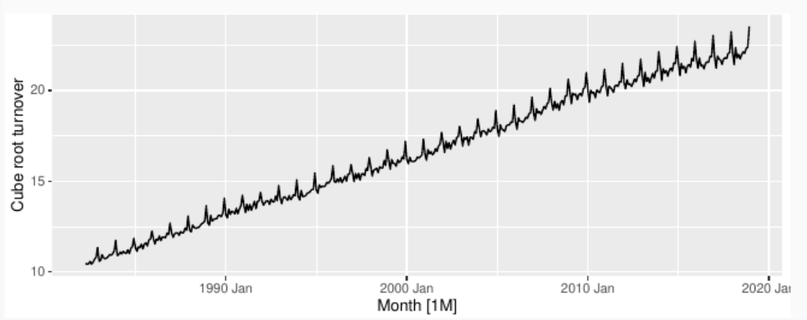


Square root

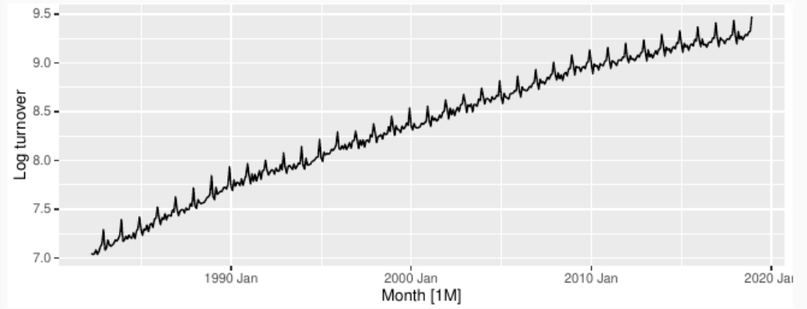
1. Distributional transformations

In general, transformations can

- be used to **normalize** the data and make it more suitable for modeling and analysis.
- **stabilize** the variance of the data and reduce the impact of outliers.
- improve the accuracy of statistical models and predictions by reducing the impact of non-normality in the data.



Cube root



Log

1. General distributional transformations

Box-Cox transform

$$w_t = \begin{cases} \log(y_t), & \lambda = 0 \\ \left(\text{sign}(y_t)\,|y_t|^{\lambda} - 1\right)/\lambda, & \lambda \neq 0. \end{cases}$$

For different choices of λ we get different transformations:

λ = 1: (No substantive transformation) λ = 1/2 : (Square root plus linear transformation) λ = 0: (Natural logarithm) λ = −1: (Inverse plus 1)

Yeo–Johnson transform

$$y_i^{(\lambda)} = \begin{cases} \left((y_i+1)^{\lambda} - 1\right)/\lambda & \text{if } \lambda \neq 0, y \geq 0 \\ \log(y_i+1) & \text{if } \lambda = 0, y \geq 0 \\ -\left((-y_i+1)^{(2-\lambda)} - 1\right)/(2-\lambda) & \text{if } \lambda \neq 2, y < 0 \\ -\log(-y_i+1) & \text{if } \lambda = 2, y < 0 \end{cases}$$

Allows also for zero and negative values

λ = 1: Identity

2. Differencing

Why?

- make patterns in the series more similar
- decreases autocorrelation in the series, increasing the ability to learn from data

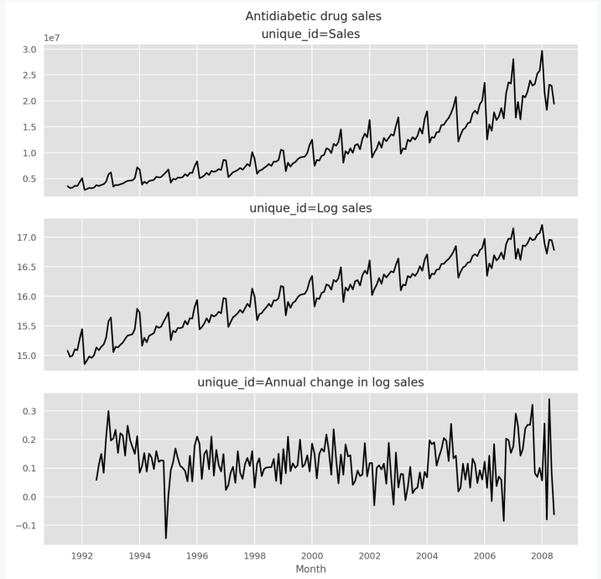
How?

- first / second order differencing

We have seen that the random walk process

$$x_t = x_{t-1} + \epsilon \quad \epsilon \sim \mathcal{N}(\prime, \infty), \quad \epsilon_{||} \perp \epsilon_{|}$$

can be made stationary by taking the first-order time derivative. ***Are there other methods to "regularize" a time series, so that we can better learn to predict it?***



Screenshot 2026-04-28 at 09.25.10.png

3. Embeddings as detrending method

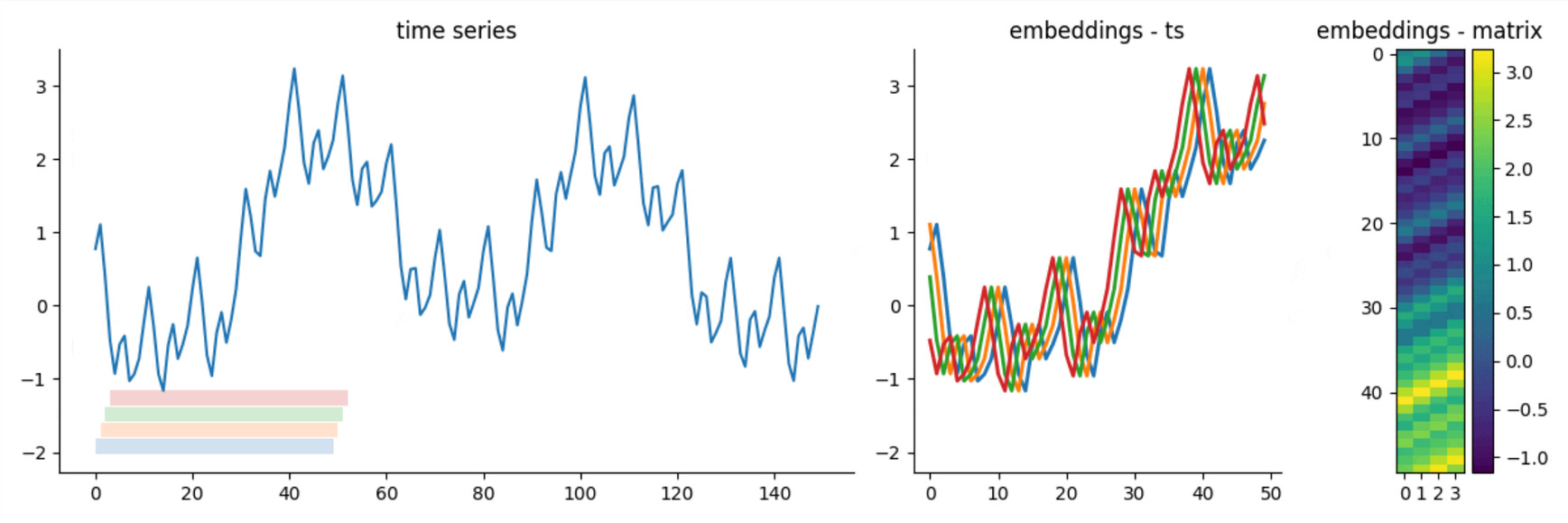
A general pattern in univariate time series forecasting is to try to learn the time series dynamics from an embedding of past values. Time embedding is the fundamental technique used by

- exponential smoothing
- state-space models
- AR, ARMA, ARIMAX models
- stateless models (generic regressors)

Time embedding for forecasting applications is mostly based on ideas from chaotic systems and system identification.

Taken's and Hankel matrices

A univariate time series $[x_t]_{t=1}^{T}$, $x \in \mathbb{R}$, can be time-embedded in a $(m, T - m\tau)$-dimensional matrix using an embedding lag τ

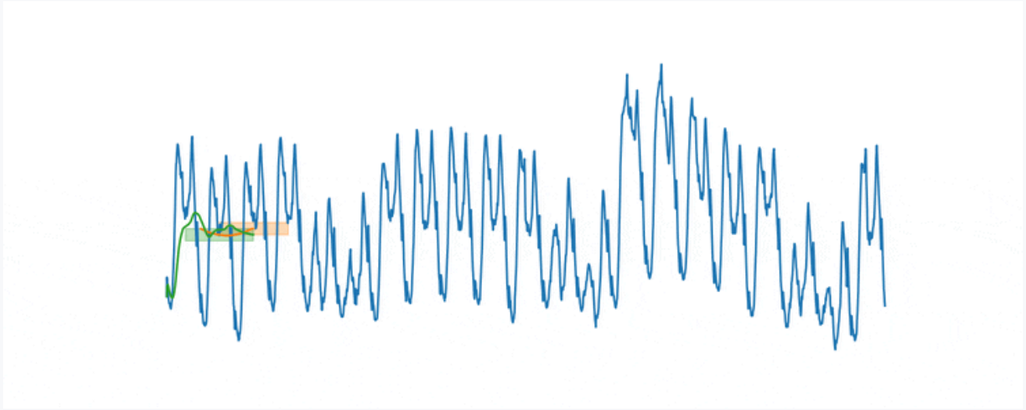


$$E(m,d) = \begin{bmatrix} x(T) & x(T-\tau) & \cdots & x(T-(m-1)\tau) \\ x(T-1) & x(T-\tau-1) & \cdots & x(T-(m-1)\tau-1) \\ x(T-2) & x(T-\tau-2) & \cdots & x(T-(m-1)\tau-2) \\ \vdots & \vdots & \ddots & \vdots \end{bmatrix}$$

Moving averages and nonanticipativity

Besides considering the last $m, \tau-$lagged values of the time series, we can also apply transformations to the embedding matrix $E(m, d)$. For $\tau = 1$, taking the rows' mean of $E(m, 1)$ computes the backward moving average of the signal.

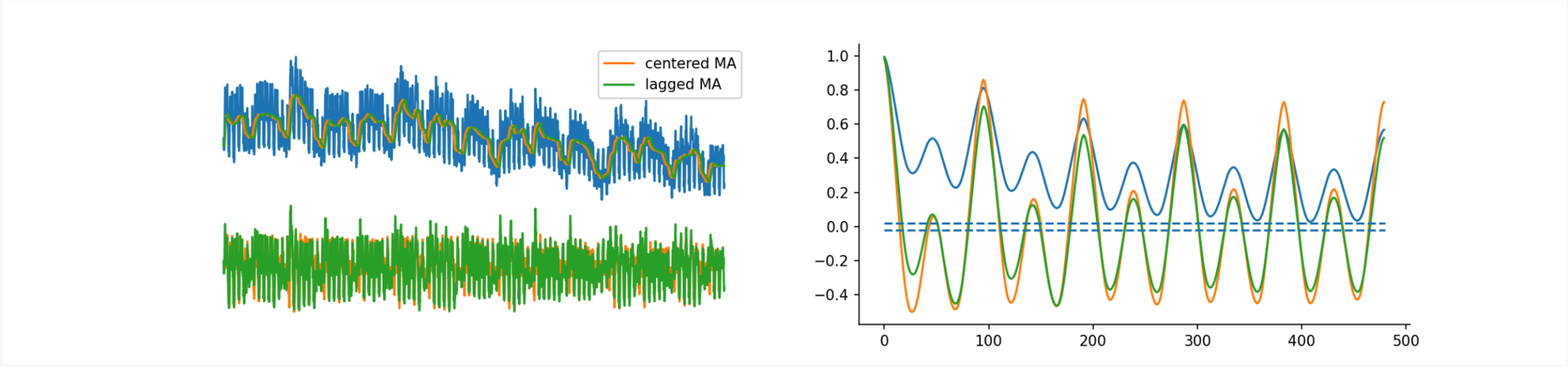
❓ Compare the centered (orange) and backward (green) 1-day moving average of the following signal. Which one can be used to forecast it?



Centered (orange) and lagged (green) 2-days moving averages for a the power consumption of a Swiss DSO.

Moving averages and nonanticipativity - 2

If the time series shows trends, a simple moving average can help making the series stationary, effectively un-correlating x_t with its past and future lags
$\rightarrow$we can better estimate moments of x_t distribution, and its joint CDF
$F_{t_1,t_2,\cdots,t_s}(b_1, b_2, \cdots, b_s) = \mathrm{P}\left(x_{t_1}(\omega) \leq b_1, x_{t_2}(\omega) \leq b_2, \ldots, x_{t_s}(\omega) \leq b_s\right)$
.



Effect of daily detrend on the ACF: the process becomes more stationary as lagged steps decorrelate

STL decompositions

STL decomposition is a popular method for decomposing a time series into its seasonal, trend, and remainder components. The acronym stands for "Seasonal and Trend decomposition using Loess," which refers to the technique used for estimating the components (**Locally Weighted Regression**), which enables smooth estimation of both trend and seasonal components without relying on global parametric assumptions.

Seasonal time series can be decomposed as:

$$y_t = f(S_t, T_t, R_t) = \begin{cases} y_t = S_t + T_t + R_t & \text{additive} \\ y_t = S_t T_t R_t & \text{multiplicative} \end{cases}$$

Applying the log to both sides of the multiplicative decomposition form leads to

$$log(y_t) = log(S_t) + log(T_t) + log(R_t)$$

multiplicative decomposition → simply take the logarithm before applying

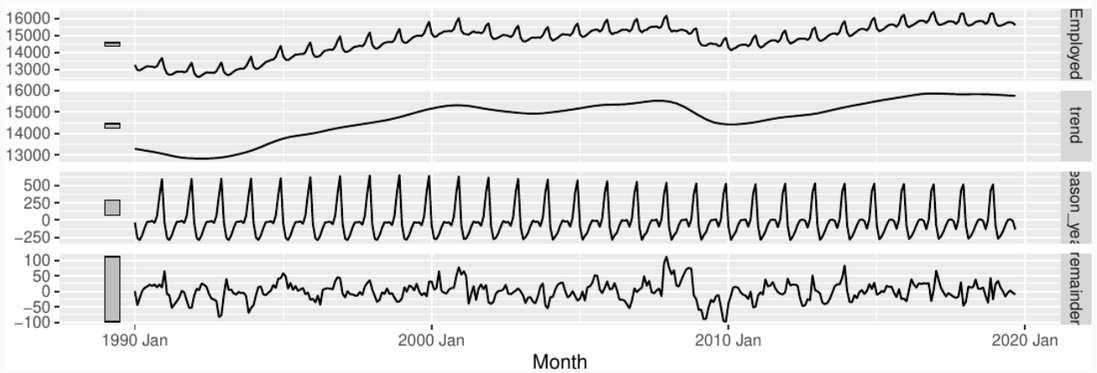
STL decompositions - 2

Pros:

- The smoothness of the trend cycle is controlled by the user.
- Robust to outliers
- Very versatile:
- trend window controls loess bandwidth applied to de-seasonalised values.
- season window controls loess bandwidth applied to detrended subseries

We just need to estimate trend T_t and seasonal components S_t

1. The seasonal component is at first obtained by averaging over all seasons
2. Trend-cycle component is obtained by fitting a loess curve to the data



Forecastability - Is there anything to model in the first place?

Forecastability is a property of the time series itself, meaning that it shouldn't depend on a forecasting model. Why is it important?

- Quick overview of task difficulty
- Should we spend time modeling the series (feature engineering) or start looking for exogenous features?
- Set expectations (with stakeholders) and promise realistic forecasting goals

Is something forecastable?

What can be forecasted?

Given a pool of signal, which ones can be **easily** forecasted?

Forecastable

- A priori: the TS shows some sort of regularity
- A posteriori: given a model m, we can predict the next value $\hat{y}_{t|t-1} = m(y_{t-1})$ with low uncertainty, as measured by appropriate KPIs, e.g. skill score w.r.t. a naive forecast:

$$S = \frac{RMSE_p - RMSE}{RMSE_p}$$

Dual question

Which signals should I **give up** forecasting?

Dual definition - Unforecastable

If the past information does not reduce the uncertainty of the future

Some a-priori measures of forecastability

We restrict ourselves to scalar measures. You can see these measures as transformations $T(y) : \mathbb{R}^T \to \mathbb{R}$ taking an univariate time series $y = [y_t]_t^T \in \mathbb{R}^T$ in input and returning a scalar.

They are attractive w.r.t. an a-posteriori forecastability test since they don't depend on the model and usually are much faster to compute (w.r.t. model training and evaluation on a proper training / test split). **However, they cannot replace the final forecasting method, which is the ultimate test for forecastability**

Variance ratios

It measures the variance of prediction errors of a naive (persistent) model with respect to the variance of the time series itself.

$$VR(k) = \frac{\text{Var}\left(y_{t+k} - y_t\right)}{\text{Var}\left(y_t\right)}$$

Autocorrelation

We've seen that the higher the autocorrelation, the more similar is the series with itself. It thus can be used as a measure of forecastability

$$\rho_k = \frac{cov(x_t, x_{t-k})}{\sigma^2(x)} \rho_K = \frac{1}{K} \sum_{k=1}^{K} |\rho_k|$$

Approximate entropy

Since the information contained in two independent events $x \perp y$ will sum, while their joint probability is $p(x, y) = p(x)p(y)$, the only possible form of the information is:

$$\log_b p(x)$$

If we take the expected value, and consider events with low probability to be more informative $\rightarrow$

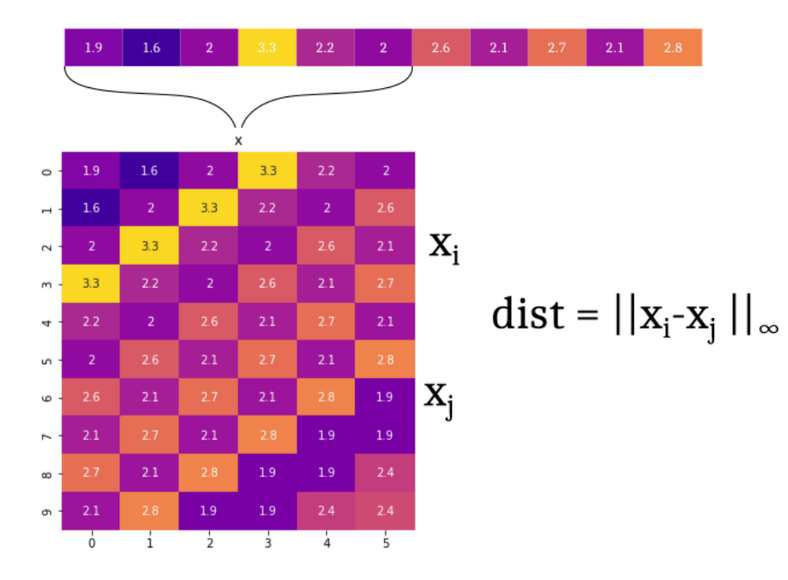
$$H = -\sum_{x \in \mathcal{X}} p(x) \log_b p(x)$$

Entropy gives an idea of complexity of the distribution of the signal.
However, entropy for these two signals will be the same:

[0,1,0,1,0,1,0,1,0,1,0,1,0,1,0,1…] [0,1,1,1,0,0,0,1,0,1,0,0,0,1,0,1, 1,…]

Approximate entropy - 2

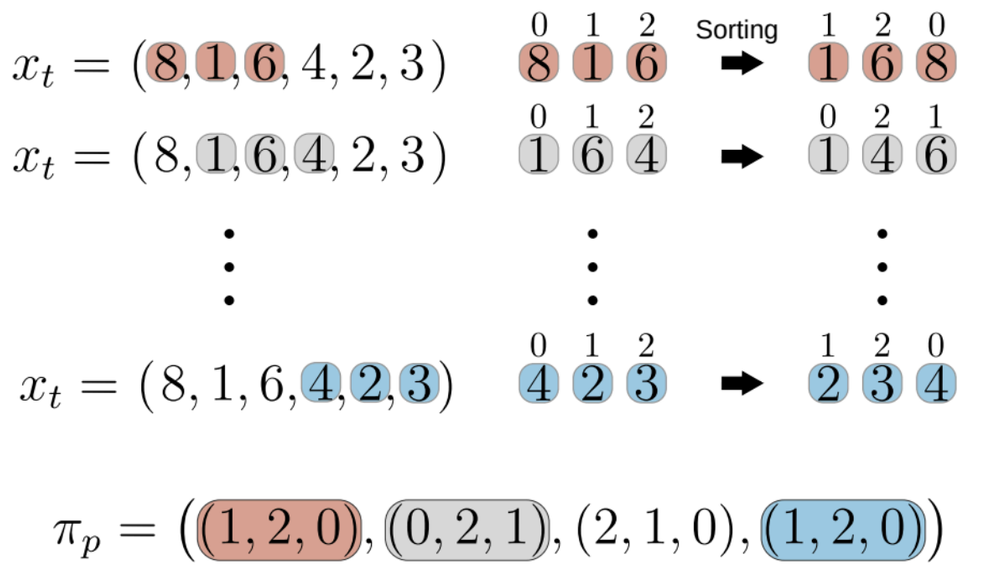
A better measure of forecastability is approximate entropy, which computes the entropy of maximum discrepancy of segments of length **m** of a time series.



$$\phi^m(r) = \frac{1}{n}\sum_{i=1}^{n} \log\left(\mathbf{1}_{dist<r}\right)$$

entropy on distribution of “r-close neighbors” of the embedded sequences

Permutation Entropy



For each subsequence of length **m**, compute the sorting permutation **π** (or ranks of the observations).

$$H_{\text{perm}} = -\frac{1}{\log(m!)} \sum_{\pi} p(\pi) \log p(\pi)$$

Lempel–Ziv complexity

This is a measure of **sequential compressibility**, starting point for some algorithms for data compression

- Start at the current position and **grow** a substring one symbol at a time.
- If the substring **has appeared before** anywhere in the already-parsed part, keep growing.
- The moment the substring would become **new** (never seen before), **cut** there — that chunk is a **new phrase**.
- Continue from the next position and repeat until the end.

Key concepts

- Distributional transformations and local transformations can increase forecastability
- Temporal embeddings are a reshape of the original time series into an Hankel matrix
- Forecastability indexes can help characterise the time series